Handwritten-Digit Recognition on an Arduino Uno

A step-by-step team plan: porting a neural network from an FPGA to an ATmega328P with a 7-segment display

Reference project: https://github.com/Si7li/MNIST-Neural-Network-On-FPGA. Contents: 1 Feasibility · 2 Team & ways of working · 3 Materials · 4 Tech stack · 5 Architecture · 6 Step-by-step plan · 7 Training the model · 8 Putting the model on the Uno · 9 Unit tests & testing · 10 Docker · 11 Pitfalls · 12 Stretch goals · 13 Resources · Appendices.

1 · Is it possible? Yes.

The FPGA project is small: a **64 → 32 → 10** network (8×8 digit images in, 10 scores out) with **2,368 int8 weights**. That is about 2.4 KB. The Uno has 32 KB of flash and runs at 16 MHz. The Uno cannot do the maths in parallel, but it does not need to.

Analogy. The FPGA is 32 cooks chopping at the same time. The Uno is one cook doing all ~2,400 chops one after another. The cook is far slower (milliseconds instead of microseconds), but the recipe is so short that a human still sees the answer instantly.

	FPGA project (Nexys 3)	This plan (Arduino Uno)
Compute	32 parallel multiply-accumulate units, 99 clock cycles	One CPU, a sequential loop of ~2,400 multiply-adds
Time per digit	≈ 8 µs (from the repo)	estimated 5–15 ms; **we will measure it**
Weight storage	ROM built into the logic	PROGMEM in flash (≈ 2.5 KB of 32 KB)
Working memory	registers	≈ 100–400 B of 2 KB SRAM
Language	VHDL	C (avr-gcc)
Input	UART from a PC drawing app	USB serial from a PC drawing app
Output	7-segment display + LEDs	single-digit 7-segment display

Dry run already done (in my sandbox, not on a real board). I trained a 64 → 32 → 10 network on the scikit-learn digits data, quantized it to int8 with int32 accumulators, and wrote the C inference core. Results on 360 held-out images: float accuracy **96.7%**, int8 accuracy **96.7%**; the C code matched the Python integer model on **360 / 360** images (identical scores). Compiled for the ATmega328P without the Arduino core it needs **3,094 B flash (9.4%)** and **104 B static RAM (5.1%)**. The reference repo reports 97.5%; the gap likely comes from a different training run and split.

Two things to check first. (1) I could read the repo's README but not its source files, so every detail below that borrows the repo's choices (pixel range, shift, rounding, UART frame) is verified in Phase 1. (2) The repo page shows no LICENSE file. Without a license, copying its code is not clearly allowed. Plan: **re-implement the idea in our own code**, credit the repo, and optionally ask the author (open an issue) for permission.

2 · Team and ways of working

Roles (one person can hold two; with 3 people, merge ML + DevOps, Firmware + Hardware, and App + QA):

Role	Owns	Main deliverables
ML / quantization	Training, integer “golden model”, export	train.py, quantize.py, export_arduino.py, accuracy report
Firmware	C inference core, serial parser, display driver	nn_core, protocol, seg7, main.cpp, native unit tests
Hardware bring-up	Wiring, segment test, timing measurements	wiring photo + pin map, bench test results

Role	Owns	Main deliverables
PC app	Drawing tool, serial client, batch sender	draw_digit.py, send_image.py, hil_test.py
QA / test	Test plan, hardware-in-the-loop runs, bug log	docs/test-report.md, pass/fail table
DevOps	Docker, Makefile, CI, README	Dockerfiles, docker-compose.yml, GitHub Actions

How the team works together

- **Contracts first.** Like agreeing on the plug shape before one person builds the plug and another the socket. On day 1, freeze: tensor shapes, fixed-point scheme, serial frame (Appendix A), pin map, repo layout.
- **One golden model.** The Python integer model is the truth. Every other piece (C code, board) must match it bit for bit.
- **Git rules.** Protected main, short feature branches, one reviewer per pull request, CI must be green to merge.
- **One card per step** of Section 6 on a shared board, each with its "done when" line.
- **Hardware sharing.** Ideally one Uno + display rig per 1–2 people. With fewer rigs, host-side tests (Section 9) let everyone work without touching hardware.
- **15-minute sync** a few times a week: blocked? contract change? who needs the rig?

3 · Materials

Item	Qty	Purpose	Status
Arduino Uno R3 (ATmega328P)	1 per rig	Runs the network	Have
Single-digit 7-segment LED display (common cathode or common anode)	1 per rig	Shows the predicted digit	Have
USB-A to USB-B cable that carries **data** (not charge-only)	1 per rig	Power, flashing, serial link to the PC	Check
Resistors 220–330 Ω, ¼ W	8 per rig	One per segment (a–g) + decimal point. ≈ 9–14 mA per segment	Need
Breadboard (half-size is enough)	1 per rig	Wiring	Need
Male-to-male jumper wires	≈ 10 per rig	Wiring	Need
Multimeter with diode test (or 3 V coin cell + resistor)	1 per team	Find the common pin and whether the display is cathode or anode	Optional
Push button	1 per rig	Optional: run the built-in self-test (cycle 0–9)	Optional
PC: Linux / macOS / Windows, Python 3.11+, Docker, free USB port	1 each	Train, build, test, draw digits	Each member

Clone boards with a CH340 USB chip work too. They may need a driver on Windows/macOS and appear as /dev/ttyUSB0 on Linux instead of /dev/ttyACM0.

Wiring (direct drive, no driver chip needed)

Segment	a	b	c	d	e	f	g	dp (optional)	Common pin
Uno pin	D2	D3	D4	D5	D6	D7	D8	D9	GND (common cathode) or 5 V (common anode)

- Put a **220–330 Ω resistor in series with every segment pin**. Never connect a segment straight to a pin.
- Avoid D0 and D1: they carry the USB serial link.
- Display pinouts differ between parts. Check its datasheet, or find the common pin and polarity with the multimeter diode test.

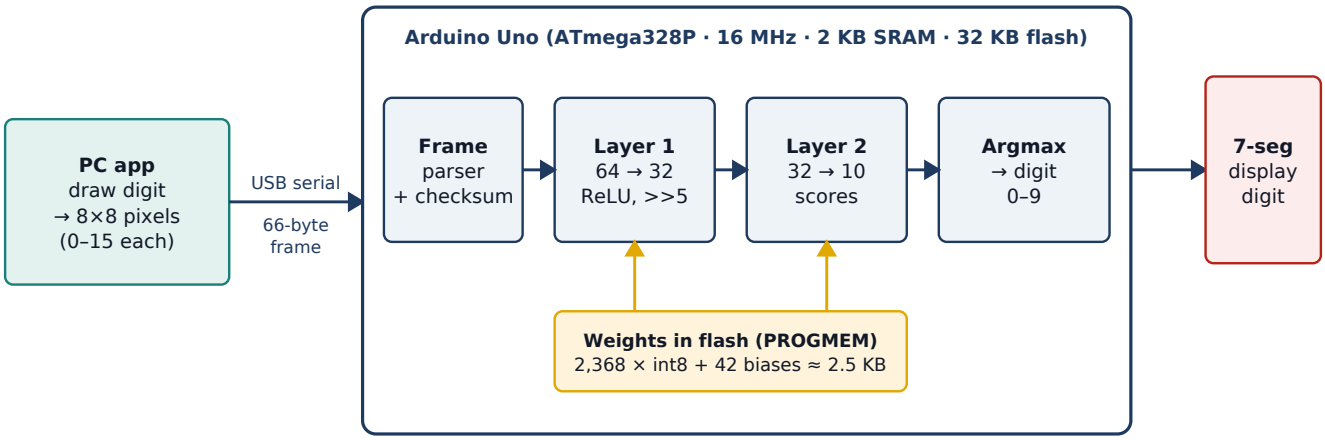
- Common cathode: a segment lights when its pin is HIGH. Common anode: it lights when LOW. The firmware has one flag for this.
- The decimal point can double as a “busy” light while a frame is received and computed.

4 · Tech stack

Layer	Tool	Why
Training	Python 3.11, PyTorch (CPU) + NumPy; scikit-learn for the dataset	Same approach as the repo. For a lighter Docker image, scikit-learn’s MLPClassifier also works (my dry run used it).
Quantization + export	NumPy script (integer golden model) that writes weights.h and test vectors	One source of truth for weights and expected answers
Firmware	C/C++ on the Arduino core, avr-gcc, **PlatformIO** (atmelavr, board uno)	One tool for build, test, flash; works the same inside Docker
Inference core	Pure C file with no Arduino includes (nn_core.c)	Compiles on a PC for unit tests and on the AVR for the board
Unit tests	pytest (Python), Unity via PlatformIO *native* environment (C)	Fast tests with no hardware
PC app	Python, Tkinter, pyserial	Same stack as the repo’s app
Flashing	avrdude (PlatformIO calls it for you)	Standard Uno uploader
Containers + CI	Docker, Docker Compose, Make, GitHub Actions	“git clone & make all” reproduces everything
Optional simulator	simavr	Run the firmware without a board (extra harness work, see Section 12)

Why not TensorFlow Lite Micro? It needs far more RAM than the Uno’s 2 KB. A small hand-written C loop is the right tool here, just as the repo hand-built its hardware instead of using vendor IP.

5 · Architecture and data flow



Stop-and-wait protocol: the PC sends one frame, waits for the Uno’s reply (digit, all 10 scores, time taken), then sends the next. Network: 64 inputs (4-bit pixels) → 32 hidden neurons with ReLU → 10 output scores → the biggest score is the digit.

6 · Step-by-step plan

After Phase 1, Phases 2, 3, 5 and 8 can run in parallel. Estimates are rough: about **2 weeks** of calendar time for 3–5 people working part-time.

Phase · owner	Tasks	Done when	Est.
P0 Kickoff All	• Create repo (layout in Section 10), task board, branch rules • Freeze contracts: shapes, fixed-point scheme, serial frame, pin map, file formats • Everyone installs Docker + Python	contracts.md merged; every member can run `make test`	0.5 d
P1 Study + spec ML + Firmware	• Read the repo README, python/train_quantize.py, app/draw_digit.py, the UART frame receiver in fpga/src • Record exact choices: pixel range, Q-format, shift, rounding, bias width, tie rule, frame format • Hand-compute one image end to end (pixels → hidden → scores → digit)	spec.md with the worked example, reviewed by 2 people	1 d
P2 Hardware bring-up Hardware	• Find common pin + polarity • Wire per Section 3 • Sketch: cycle digits 0–9, then all segments on	All segments and digits 0–9 look right; wiring photo + pin map committed	0.5–1 d
P3 Train, quantize, export ML	• train.py: fixed seed, stratified 80/20 split • quantize.py: integer golden model with the firmware's exact arithmetic • export_arduino.py: weights.h + golden vectors (inputs and expected scores) • Report: float vs int accuracy, confusion matrix, activation ranges (to pick the shift)	Int accuracy within 1 point of float; two exports give the same hash	1–2 d
P4 Firmware core + host tests Firmware	• nn_core (pure C, PROGMEM reads behind a macro), seg7, protocol parser • Unity tests (Section 9, levels 2–4) on the native environment	100% golden match on the host; tests green in CI	1–2 d
P5 PC app App (+ Firmware)	• Drawing canvas → 8×8, 0–15, same preprocessing as the training data • Serial client: wait ~2 s after opening the port (the Uno resets), stop-and-wait, timeout + retry • send_image.py (CLI) and hil_test.py (batch)	App shows the Uno's reply: digit, scores, microseconds	1–2 d
P6 On-board integration Firmware + Hardware	• Flash the board; run bench steps B1–B3 • Read the RAM/flash numbers from the build; measure time per digit	Known-answer images show the correct digit on the display	1 d
P7 Test campaign QA	• Bench steps B4–B7; log in docs/test-report.md • File bugs for any mismatch • Live trial with several people's handwriting	Hardware-in-the-loop: 0 mismatches; soak test clean; report written	1–2 d
P8 Docker + CI DevOps (from P1)	• Dockerfiles, compose file, Makefile; pin every version • Bake the PlatformIO toolchain into the image • GitHub Actions: tests + size budget + "regenerated weights equal committed weights" • Clean-machine test by someone new to the project	`git clone && make all` gives the same hash and .hex on a fresh PC	1–2 d
P9 Docs + demo All	• README with a ≤ 10-command quick start • Results table: accuracy, time per digit, flash/RAM; comparison with the FPGA • Demo GIF; credit the reference repo	A new person reproduces the project from the README alone	1 d

7 · Training the model

Resources

- **Data:** scikit-learn *digits*, 1,797 images of 8×8 pixels. It ships inside scikit-learn, so there is no download and Docker builds stay offline and reproducible.
- **Compute:** any laptop CPU. The repo reports about 7 s of training. No GPU, no Colab.
- **Model:** 64 → 32 (ReLU) → 10 = 2,368 weights + 42 biases = 2,410 parameters.
- **Recipe:** fixed seeds, stratified 80/20 split (360 test images, same count as the repo), Adam optimizer, CPU only.

- **Hand-drawn digits are different from the dataset.** Mouse strokes are thicker and placed differently. Fix both sides: preprocess drawings like the dataset (crop, centre, average-pool to 8×8, scale to 0–15) and add small shifts as augmentation.

Quantization scheme (proposal, from my dry run)

Weights become int8 with 6 fractional bits (scale 64). Pixels stay integers 0–15. Sums use **int32**. After layer 1: ReLU, shift right, clamp to 0–127. The digit is the first maximum score (NumPy's argmax rule).

python/quantize.py (core of the golden model)

```
S, SH = 64, 5                                       # weight scale (Q2.6), hidden-layer shift
w1 = np.clip(np.rint(W1 / 15.0 * S), -128, 127).astype(np.int32)   # model trained on x/15
b1 = np.rint(b1_float * S).astype(np.int32)
w2 = np.clip(np.rint(W2 * S), -128, 127).astype(np.int32)
b2 = np.rint(b2_float * 2 * S).astype(np.int32)    # hidden scale is S / 2**SH = 2

def golden(x):                                      # x: ints 0..15, shape (N, 64)
    a1 = x @ w1.T + b1
    h = np.clip(np.maximum(a1, 0) >> SH, 0, 127)
    return h @ w2.T + b2                            # int32 scores; digit = argmax
```

- **Why int32:** worst case layer 1 is 64 × 15 × 128 ≈ 123,000, far above the int16 limit of 32,767. On AVR a plain `int` is only 16 bits.
- **Tune the shift.** In my dry run the hidden values peaked at 14, so a shift of 5 wastes most of the 0–127 range, and layer-1 weights only spanned −5…+6 because the model was trained on normalised inputs. Accuracy still held, but pick the shift from the activation histogram (Phase 3), and match what the repo does (Phase 1).
- **Stretch idea:** full 28×28 MNIST (60k training / 10k test images, via torchvision or OpenML). A 784 → 16 → 10 net is ≈ 12.7 KB of weights and fits in flash, but 784 pixels do not fit comfortably in 2 KB of RAM, so layer 1 would accumulate pixels as they stream in. Expect lower accuracy than the 8×8 version; measure it.

8 · Putting the trained model on the Uno

The model is not "loaded" like a file. It is compiled into the firmware as constant arrays in flash. A new model means a new `weights.h` and a re-flash.

Steps

- **1. Export.** `python python/export_arduino.py` writes `firmware/include/weights.h` (arrays W1[2048], B1[32], W2[320], B2[10] in PROGMEM) and the golden vectors for host tests only (not flashed).
- **2. Read weights the AVR way.** Flash and RAM are separate address spaces on the Uno. Reading a PROGMEM array with a normal pointer returns garbage, so use `pgm_read_byte` / `pgm_read_word` and cast to a signed type.
- **3. Build.** `pio run -e uno`. Check the printed Flash/RAM percentages against the budget in Section 9.
- **4. Flash.** Close the PC app first (only one program can hold the serial port), then `pio run -e uno -t upload --upload-port /dev/ttyACM0`.
- **5. First light.** The display shows "-" while waiting. Run the bench steps in Section 9.
- **Troubleshooting:** permission denied → add yourself to the *dialout* group (*uucp* on Arch); wrong port → `pio device list`; upload hangs → wrong board or port busy.

platformio.ini

```
; firmware/platformio.ini
[env:uno]
platform = atmelavr          ; pin an exact version once the team picks one
board = uno
framework = arduino
monitor_speed = 115200

[env:native]                  ; runs unit tests on the PC, no board needed
platform = native
test_framework = unity
```

main.cpp

```
// firmware/src/main.cpp (skeleton)
#include <Arduino.h>
extern "C" {
#include "nn_core.h"
}
#include "seg7.h"
#include "protocol.h"

static uint8_t img[NN_IN];
static int32_t scores[NN_OUT];

void setup() {
  seg7_init();                // pins D2..D8, common-anode flag lives in seg7.h
  seg7_show_dash();           // "-" means waiting
  Serial.begin(115200);
}

void loop() {
  if (!protocol_read_frame(img)) return;     // non-blocking; timeout resets the parser
  seg7_set_dp(true);                         // busy light
  uint32_t t0 = micros();
  uint8_t digit = nn_infer(img, scores);
  uint32_t dt = micros() - t0;
  seg7_show(digit);
  protocol_reply(digit, scores, dt);         // see Appendix A
}
```

9 · Unit tests and how to test

Test levels (bottom = fastest, no hardware)

Level	What it checks	Tool · where	Pass when
1 · Python unit	Weights fit in int8; shapes (32×64, 10×32); same seed gives the same weights.h hash; int accuracy within 1 point of float	pytest · Docker	All green
2 · C core (host)	ReLU/clamp edge cases; worst-case inputs do not overflow (all pixels 15 × weight −128); argmax tie picks the lowest index; **every golden vector gives identical scores**	Unity, PlatformIO native · Docker	0 mismatches
3 · Protocol	Good frame accepted; bad checksum rejected; truncated frame times out; garbage before the header resyncs; pixel above 15 handled as the spec says	Unity native · Docker	All green
4 · 7-segment driver	Table for 0–9 correct (Appendix B); common-anode output is the inverse; uses a mock pin writer	Unity native · Docker	All green
5 · Build budget	Parse the avr-size output of the build	Script in CI · Docker	Flash ≤ 50%, static RAM ≤ 60% (keeps stack headroom)
6 · On-board test (HIL)	hil_test.py sends all golden vectors over USB; compares digit and all 10 scores; records time per digit	Python · real board	100% match
7 · Simulator (optional)	Run the .elf in simavr with a small harness	simavr · Docker	Same as level 6
8 · Acceptance	Team members draw 30 digits live	PC app · real board	Team-agreed target, e.g. ≥ 80% (hand-drawn is below the 97% test-set figure)

Bench procedure on the board (do in order)

Step	Test	Pass when
B1	Power + display: cycle 0–9 every 500 ms, then all segments on	Every segment lights; digits look right
B2	Serial echo: PC sends a frame, Uno answers with the checksum	Reply matches; no framing errors
B3	Known answer: send one golden image per digit	Display shows the right digit each time
B4	Batch: hil_test.py with all 360 vectors	0 mismatches in digit and scores
B5	Timing: 100 runs, min / median / max microseconds	Recorded (my estimate: 5–15 ms)
B6	Soak: 1,000 frames back to back	No hangs, no resets, same answers
B7	Live demo: draw digits in the app	Answers shown on the display; note the hit rate

10 · Making it reproducible with Docker

Goal: on any machine, `git clone && make all` gives the same weights, the same .hex and the same test results.

What runs where

Task	Container?	Notes
Train, quantize, export	Yes (train image)	CPU only, seeds fixed
Unit tests (levels 1–5)	Yes	No hardware needed
Build the .hex	Yes (firmware image)	PlatformIO toolchain baked into the image
Flash + hardware tests	Yes on Linux, pass the device in	`--device /dev/ttyACM0`. Docker Desktop on macOS/Windows cannot pass USB serial: build in Docker, flash from the host (or use usbipd with WSL2)
Drawing app (Tkinter)	Run on the host	GUI forwarding from containers is fiddly. Docker still has the headless send_image.py

Suggested repo layout

```
arduino-mnist-nn/
├─ python/        train.py  quantize.py  export_arduino.py  tests/
├─ firmware/      platformio.ini  src/main.cpp  include/weights.h (generated)
│                 lib/nn_core  lib/protocol  lib/seg7  test/
├─ app/           draw_digit.py  send_image.py
├─ tools/         hil_test.py  size_check.py
├─ docker/        Dockerfile.train  Dockerfile.firmware
├─ docs/          spec.md  contracts.md  wiring.jpg  test-report.md
├─ docker-compose.yml   Makefile   README.md
```

Dockerfile (sketch, not yet run)

```
# docker/Dockerfile.firmware
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends gcc make git \
    && rm -rf /var/lib/apt/lists/*
RUN pip install --no-cache-dir platformio==<pinned-version>
WORKDIR /work
COPY firmware/platformio.ini firmware/
# warm-up: download the AVR toolchain into the image (builds then work offline)
RUN cd firmware && pio pkg install -e uno && pio pkg install -e native
```

docker-compose.yml (sketch, not yet run)

```
# docker-compose.yml (excerpt)
x-fw: &fw
  build: { context: ., dockerfile: docker/Dockerfile.firmware }
  volumes: [".:/work"]
services:
  train:
    build: { context: ., dockerfile: docker/Dockerfile.train }
    volumes: [".:/work"]
    command: make -C python all
  test:
    <<: *fw
    command: make test
  build:
    <<: *fw
    command: make -C firmware hex
  flash:                                  # Linux hosts only
    <<: *fw
    devices: ["/dev/ttyACM0:/dev/ttyACM0"]
    command: make -C firmware upload
```

Make targets: `make train` · `make test` · `make build` · `make flash` · `make hil` · `make all`. Each one just calls `docker compose run` for the matching service.

Reproducibility rules

- Pin everything: base image tag, `pip` lock file, PlatformIO and platform versions. Fixed seeds, CPU-only training, deterministic mode.
- Commit the generated `weights.h` (a few KB) so firmware builds do not need training. CI retrains and fails if the result differs from the committed file.
- PlatformIO downloads toolchains at first build; baking them into the image avoids surprises.
- Final check: a teammate on a clean machine runs the README commands and compares the .hex hash.

> **NixOS hosts:** enable Docker with `virtualisation.docker.enable = true;` and add your user to `extraGroups = [ "docker" "dialout" ];`. For the host-side drawing app: `nix-shell -p "python3.withPackages (ps: [ ps.tkinter ps.pyserial ])"`.

11 · Pitfalls to expect

Pitfall	What happens	Fix
`int` is 16-bit on AVR	Silent overflow, wrong digits	Use int32_t accumulators; test with worst-case inputs
Reading PROGMEM like normal memory	Garbage weights	pgm_read_byte / pgm_read_word, cast to int8_t / int16_t
Opening the serial port resets the Uno	First frame is lost	PC app waits ~2 s after opening the port
Serial receive buffer is 64 bytes; a frame is 66	Dropped bytes if the loop is busy	Read bytes as they arrive, compute only after the full frame; stop-and-wait from the PC
115200 baud has ≈ 2% error on a 16 MHz AVR	Usually fine; rarely garbled frames	Try 57600 or 250000 if errors appear
Python and C disagree by one	Shift, rounding or tie-rule mismatch	Golden vectors on every build; compare all 10 scores, not just the digit
Hand-drawn digits fail but test set passes	Domain gap, not a code bug	Same preprocessing as training data; augmentation
Two programs want the serial port	“Port busy” on flash or app start	Close the app before flashing
Common anode vs cathode swapped	Display shows inverted segments	One compile-time flag; test with B1 first
Too much current through a pin	Hot pins, damaged chip	Always use resistors (20 mA per pin is the recommended limit, 40 mA absolute max)
Copying code from an unlicensed repo	Unclear legal footing	Re-implement, credit, ask the author

12 · Stretch goals

- **Speed:** loop over pixels and skip the zeros (most of an 8×8 digit is empty). Could give a 2–3× speed-up; measure it.
- **Full 28×28 MNIST** with a streaming first layer (Section 7).
- **Smaller or smarter weights:** 4-bit weights, quantization-aware training.
- **simavr in CI:** run the firmware in a simulator. Needs a small C harness that connects the simulated UART to the test script, so budget extra time.
- **Confidence:** blink the decimal point when the top two scores are close, or plot scores on the serial plotter.
- **Report:** a side-by-side table of FPGA vs Uno (time, resources, effort).

13 · Resources

Topic	Link
Reference project (read the README + python/ folder first)	https://github.com/Si7li/MNIST-Neural-Network-On-FPGA
scikit-learn digits dataset	https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
PyTorch (CPU install)	https://pytorch.org/get-started/locally/
PlatformIO: Atmel AVR platform	https://docs.platformio.org/en/latest/platforms/atmelavr.html
PlatformIO: unit testing	https://docs.platformio.org/en/latest/advanced/unit-testing/index.html
Arduino Uno R3 hardware docs	https://docs.arduino.cc/hardware/uno-rev3/
avr-libc: data in program space (PROGMEM)	https://www.nongnu.org/avr-libc/user-manual/pgmspace.html
simavr (AVR simulator)	https://github.com/buserror/simavr
Integer-only quantization paper (Jacob et al., 2018)	https://arxiv.org/abs/1712.05877
Docker Compose	https://docs.docker.com/compose/

Appendix A · Serial protocol (proposal; confirm against the repo in Phase 1)

115200 baud, 8N1, stop-and-wait. All multi-byte numbers are little-endian.

Direction	Bytes	Layout
PC → Uno (request, 66 B)	1 + 64 + 1	0xA5 · 64 pixel bytes (0–15 each, row by row) · XOR of the 64 pixel bytes
Uno → PC (reply, 47 B)	1 + 1 + 40 + 4 + 1	0x5A · digit (0–9) · 10 × int32 scores · microseconds taken (uint32) · XOR of all bytes between header and checksum
Uno → PC (error, 1 B)	1	0x15 on a bad checksum or invalid pixel; the PC may resend

Time on the wire at 115200 baud: request ≈ 5.7 ms, reply ≈ 4.1 ms. The parser drops a half-received frame after 100 ms without a byte.

Appendix B · 7-segment table and display logic

Bit 0 = segment a … bit 6 = segment g. For common anode, invert the byte before writing it to the pins.

Digit	0	1	2	3	4	5	6	7	8	9	dash
Byte	0x3F	0x06	0x5B	0x4F	0x66	0x6D	0x7D	0x07	0x7F	0x6F	0x40
Lit	abcdef	bc	abdeg	abcdg	bcfg	acdfg	acdefg	abc	abcdefg	abcdfg	g

Appendix C · Inference core (tested)

This is the C core from my dry run. It compiled with gcc and avr-gcc 7.3 and matched the Python golden model on all 360 test vectors. Treat it as a starting point; Phase 1 may change the shift, rounding or clamp to match the repo.

lib/nn_core/nn_core.c

```
#include "nn_core.h"
#include "weights.h"
#ifdef __AVR__
  #define RD8(p)  ((int8_t)pgm_read_byte(p))
  #define RD16(p) ((int16_t)pgm_read_word(p))
#else
  #define RD8(p)  (*(p))
  #define RD16(p) (*(p))
#endif

uint8_t nn_infer(const uint8_t *x, int32_t *scores_out) {
    uint8_t hid[NN_HID];
    for (uint8_t j = 0; j < NN_HID; j++) {
        int32_t acc = RD16(&B1[j]);                      /* int32: int is 16-bit on AVR */
        for (uint8_t i = 0; i < NN_IN; i++)
            acc += (int16_t)x[i] * RD8(&W1[(uint16_t)j * NN_IN + i]);
        if (acc < 0) acc = 0;                            /* ReLU */
        acc >>= NN_SHIFT;
        hid[j] = (acc > 127) ? 127 : (uint8_t)acc;       /* clamp */
    }
    uint8_t best = 0; int32_t best_s = 0;
    for (uint8_t k = 0; k < NN_OUT; k++) {
        int32_t acc = RD16(&B2[k]);
        for (uint8_t j = 0; j < NN_HID; j++)
            acc += (int16_t)hid[j] * RD8(&W2[(uint16_t)k * NN_HID + j]);
        if (scores_out) scores_out[k] = acc;
        if (k == 0 || acc > best_s) { best_s = acc; best = k; }   /* first max wins, like numpy */
    }
    return best;
}
```